3. Multimodal Visual Understanding + Basic Control

1. Course Content

1. Learn the large model's use of the robot's basic control functions.
2. Analyze newly appearing key source code.

2. Preparation

2.1 Content Description

> The virtual machine used by default with this product starts the Dify docker environment automatically at boot, so no additional startup is needed.

⚠ The same test instructions will not produce exactly the same reply content from the large model, and will differ slightly from the screenshots in the tutorial. If you need to enhance or reduce the diversity of the large model's replies, refer to 【AI Large Model Development】-【RAG Knowledge Base Deployment】 to modify the knowledge base and sample library.

Virtual machine terminal, start the Dify environment:

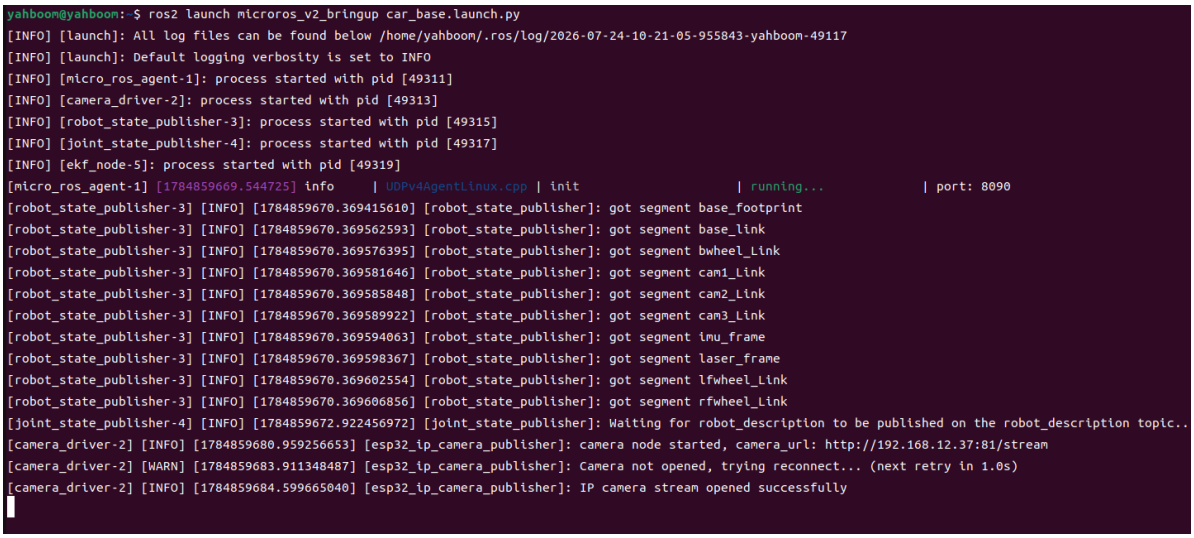
```
sh ~/bringup_dify.sh
```

3. Running the Case

3.1 Starting the Program

Open the virtual machine terminal and enter the command to start the underlying proxy + camera topic:

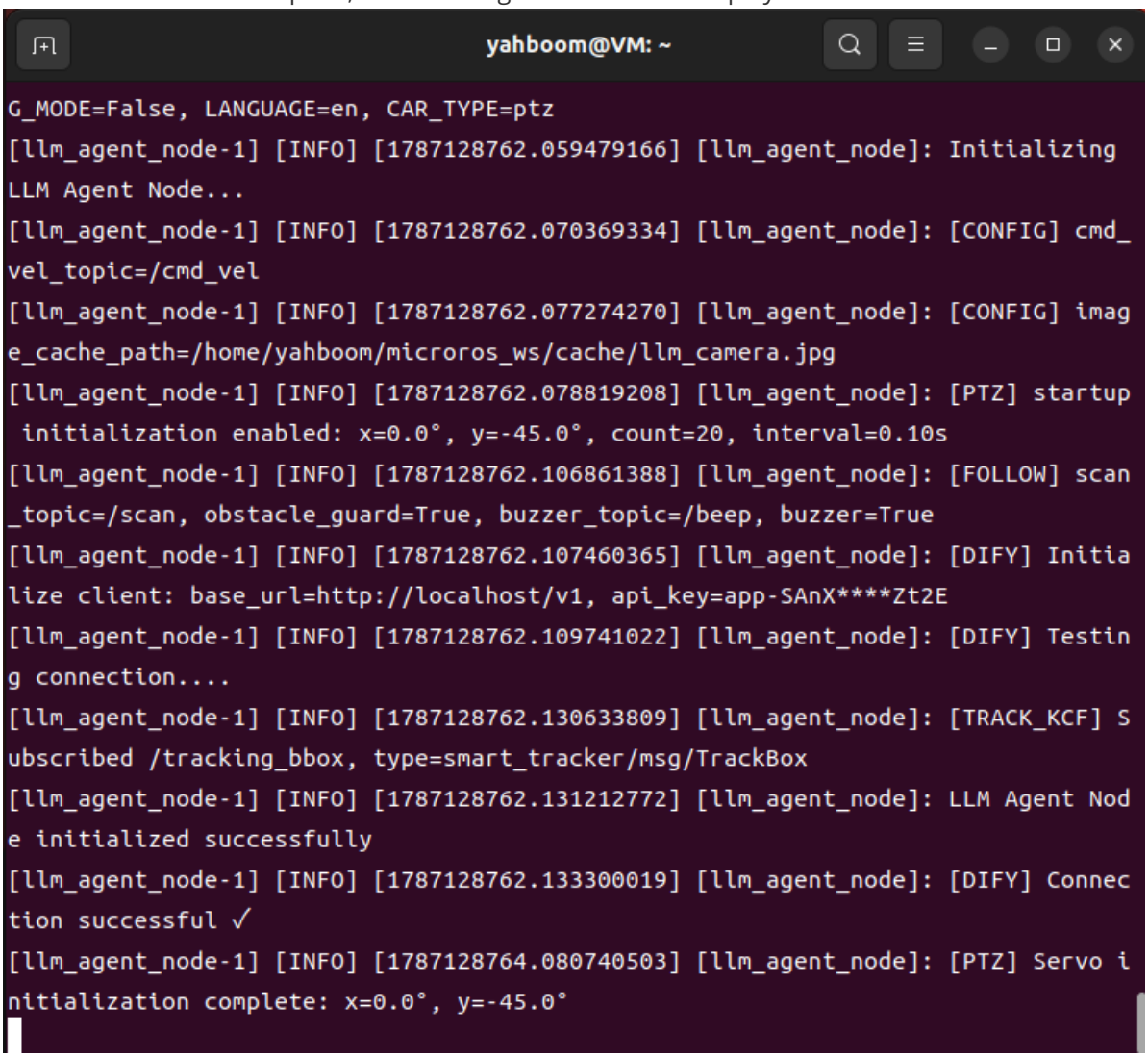
```
ros2 launch microros_v2_bringup car_base.launch.py
```



Open another virtual machine terminal and enter the command:

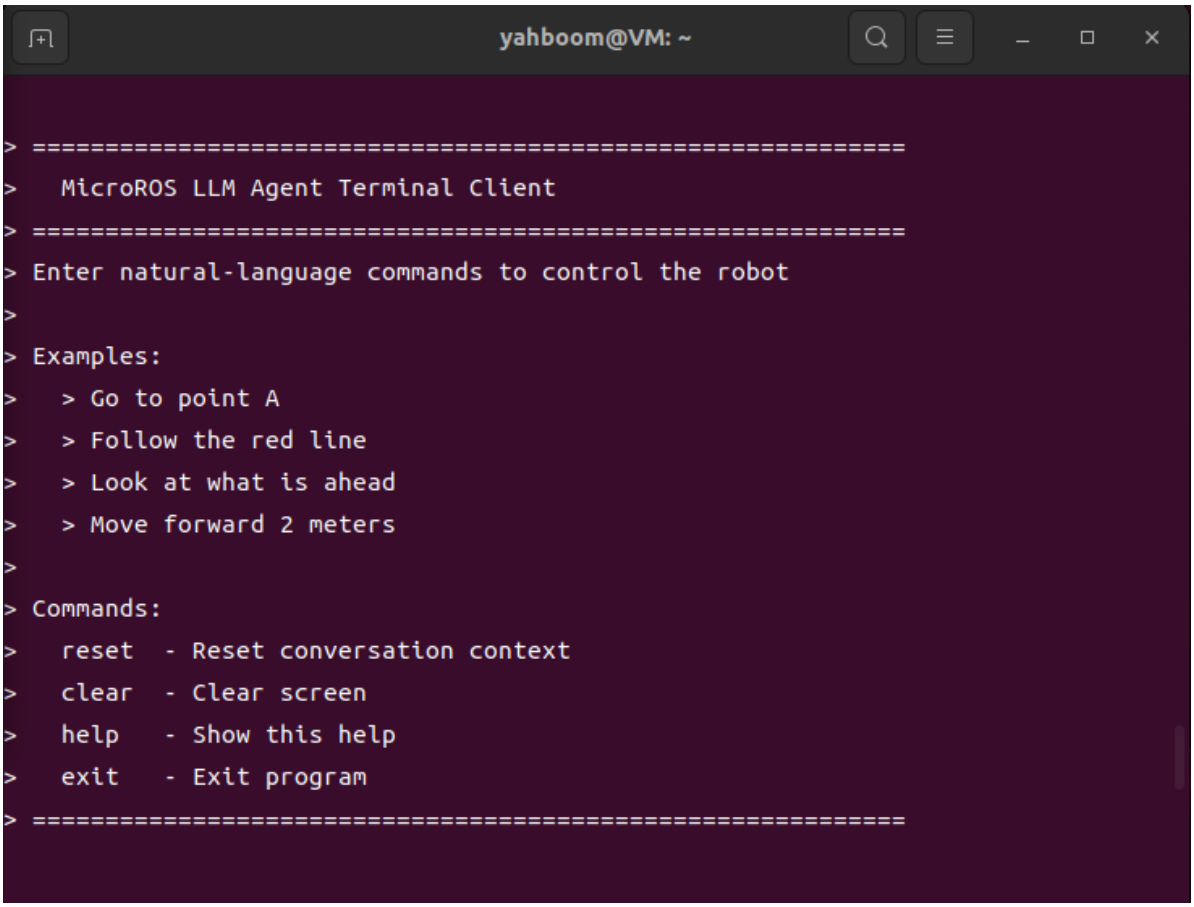
```
ros2 launch multi_brains llm_agent_control.launch.py
```

After initialization is complete, the following content will be displayed:



Then open another terminal and start the text dialogue:

```
ros2 run multi_brains terminal_client
```



3.2 Test Cases

Note: Servo 1 is the x-direction servo with a range of (-90-90), servo 2 is the y-direction with a range of (-90-0), and the initial position is (0, -45).

Reference test cases are given here. Users can create their own dialogue commands.

- The car moves forward 1 meter, backward 0.5 meters, then turns left 90 degrees, then turns right 90 degrees.
- Servo 1 rotates to 30 degrees, servo 2 rotates to -30 degrees, then returns to the initial position.
- Take photos around.

3.2.1 Case 1: "The car moves forward 1 meter, backward 0.5 meters, then turns left 90 degrees, then turns right 90 degrees"

Enter "The car moves forward 1 meter, backward 0.5 meters, then turns left 90 degrees, then turns right 90 degrees" in the text dialogue terminal. The terminal prints information as follows:

```
ecision] thinking...
[llm_agent_node-1] [INFO] [1787128972.008249597] [llm_agent_node]: [PROGRESS] [d
ecision] Starting the movement sequence: first moving forward 1 meter
[llm_agent_node-1] [INFO] [1787128972.009093295] [llm_agent_node]: [PROGRESS] De
cision plan: 1.move forward (speed 0.5, 2 seconds)
[llm_agent_node-1] [INFO] [1787128972.009980693] [llm_agent_node]: [PROGRESS] [p
reparing] adjusting camera to face forward...
[llm_agent_node-1] [INFO] [1787128972.513495566] [llm_agent_node]: [PROGRESS] [a
ction 1/1] move_forward(0.5, 2)
[llm_agent_node-1] [INFO] [1787128972.514414913] [llm_agent_node]: [MOTION] move
 forward: speed=0.5m/s, duration=2.0s, distance≈1.00m
[llm_agent_node-1] [INFO] [1787128974.523411243] [llm_agent_node]: [PROGRESS] [a
ction 1/1] move_forward(0.5, 2) succeeded ✓
[llm_agent_node-1] [INFO] [1787128977.417196281] [llm_agent_node]: [PROGRESS] [f
eedback decision] First step completed. Now moving backward 0.5 meters
[llm_agent_node-1] [INFO] [1787128977.418099108] [llm_agent_node]: [PROGRESS] De
cision plan: 1.move backward (speed 0.5, 1 seconds)
[llm_agent_node-1] [INFO] [1787128977.418826534] [llm_agent_node]: [PROGRESS] [a
ction 1/1] move_backward(0.5, 1)
[llm_agent_node-1] [INFO] [1787128977.419415703] [llm_agent_node]: [MOTION] move
 backward: speed=0.5m/s, duration=1.0s, distance≈0.50m
[llm_agent_node-1] [INFO] [1787128978.423763086] [llm_agent_node]: [PROGRESS] [a
ction 1/1] move_backward(0.5, 1) succeeded ✓
```

The robot decomposes this into four tasks to complete. After completing the tasks, it enters a waiting state.

3.2.2 Case 2: "Servo 1 rotates to 30 degrees, servo 2 rotates to -30 degrees, then returns to the initial position"

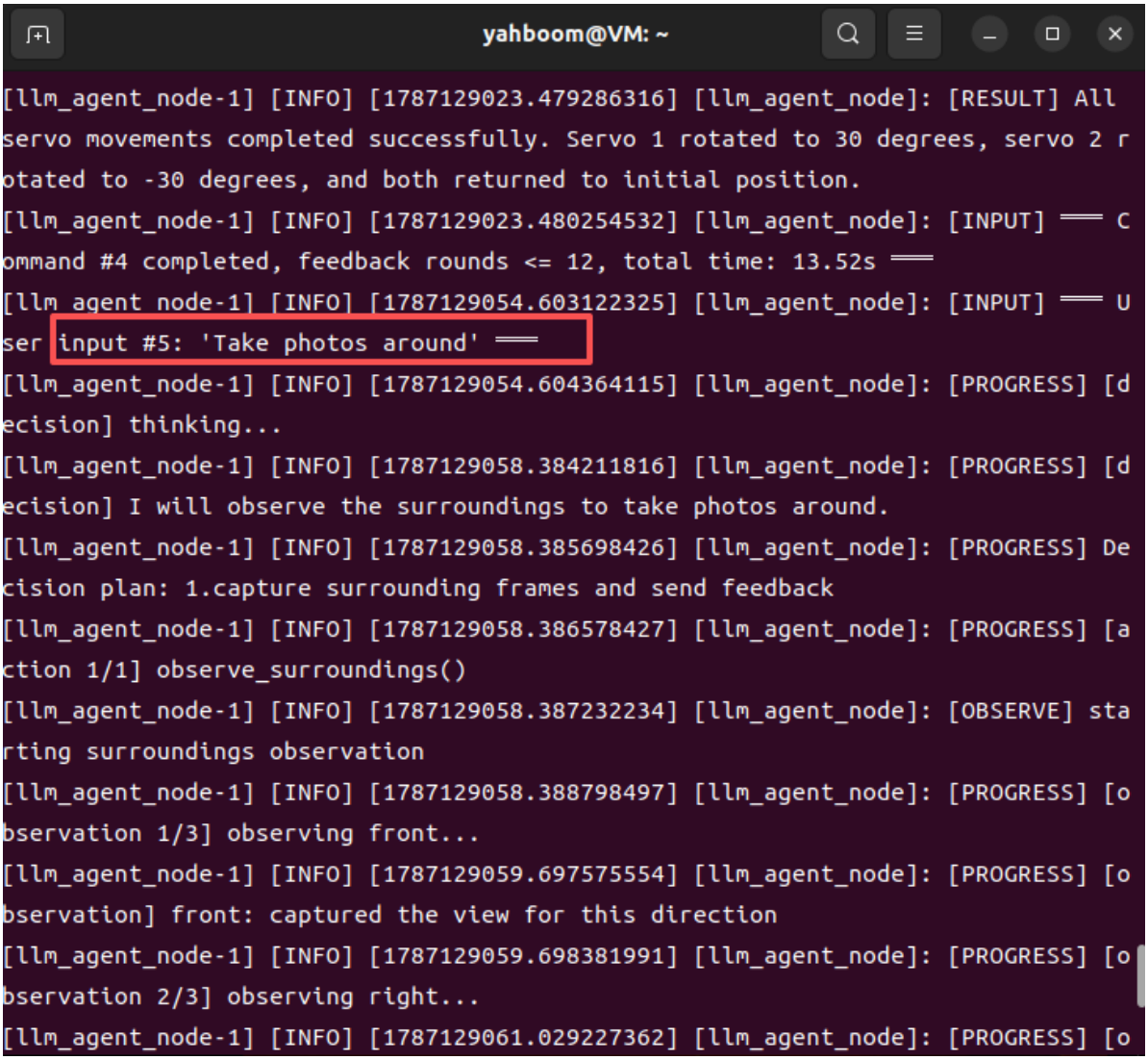
Enter "Servo 1 rotates to 30 degrees, servo 2 rotates to -30 degrees, then returns to the initial position" in the terminal. The terminal prints information as follows:

```
ction 1/1] servo2_move(-30)
[llm_agent_node-1] [INFO] [1787129017.223618025] [llm_agent_node]: [SERVO] 2 ser
vo movement complete: y=-30.0°
[llm_agent_node-1] [INFO] [1787129017.227009177] [llm_agent_node]: [PROGRESS] [a
ction 1/1] servo2_move(-30) succeeded ✓
[llm_agent_node-1] [INFO] [1787129020.049377569] [llm_agent_node]: [PROGRESS] [f
eedback decision] Servo 2 rotation to -30 degrees completed, now returning to in
itial position
[llm_agent_node-1] [INFO] [1787129020.050253883] [llm_agent_node]: [PROGRESS] De
cision plan: 1.reset PTZ servos
[llm_agent_node-1] [INFO] [1787129020.051042919] [llm_agent_node]: [PROGRESS] [a
ction 1/1] servo_init()
[llm_agent_node-1] [INFO] [1787129020.254556238] [llm_agent_node]: [SERVO] PTZ r
eset complete: x=0.0°, y=-45.0°
[llm_agent_node-1] [INFO] [1787129020.255746858] [llm_agent_node]: [PROGRESS] [a
ction 1/1] servo_init() succeeded ✓
[llm_agent_node-1] [INFO] [1787129023.478333487] [llm_agent_node]: [ACTION] fini
shtask: task cycle complete
[llm_agent_node-1] [INFO] [1787129023.479286316] [llm_agent_node]: [RESULT] All
servo movements completed successfully. Servo 1 rotated to 30 degrees, servo 2 r
otated to -30 degrees, and both returned to initial position.
[llm_agent_node-1] [INFO] [1787129023.480254532] [llm_agent_node]: [INPUT] ═══ C
ommand #4 completed, feedback rounds <= 12, total time: 13.52s ═══
```

The robot decomposes this into three tasks to complete. After completing the tasks, it enters a waiting state.

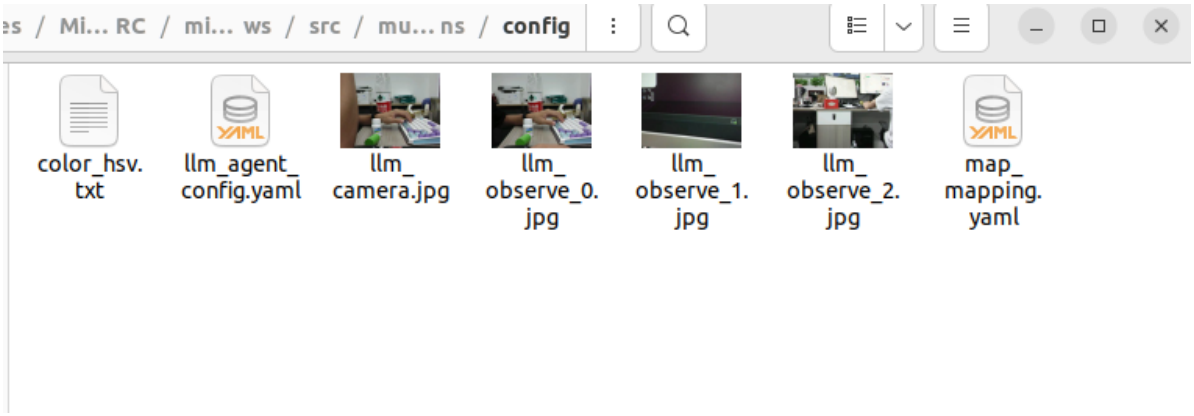
3.2.3 Case 3: "Take photos around"

Enter "Take photos around" in the terminal. The terminal prints information as follows:



You can see that the robot takes photos in three directions and saves them, then analyzes the photos taken. After completing the task, it enters a waiting state. Photo paths:

```
~/microros_ws/cache/llm_observe_0.jpg   llm_observe_1.jpg llm_observe_2.jpg
```



At this time, the instruction is directly passed to the execution-layer large model, and all conversation history is always retained. If the conversation becomes too long, we can input the "/reset" command again to let the robot end the current task cycle and start a new task cycle.

```
[llm_agent_node-1] [INFO] [1784861415.736156469] [llm_agent_node]: [INPUT] ═══
Received user input #9: '/reset' ═══
[llm_agent_node-1] [INFO] [1784861415.738477949] [llm_agent_node]: [PROGRESS]
[Decision] Thinking...
[llm_agent_node-1] [INFO] [1784861421.584339073] [llm_agent_node]: [ACTION] fini
shtask: Task cycle complete
[llm_agent_node-1] [INFO] [1784861421.585370090] [llm_agent_node]: [RESULT]
System reset, waiting for new command
[llm_agent_node-1] [INFO] [1784861421.586198997] [llm_agent_node]: [INPUT] ═══
Command #9 processing complete, feedback rounds ≤12, total time: 5.85s ═══
```

4. Source Code Analysis

This case uses the structure where the Dify execution layer outputs action functions, which are then distributed by `llm_agent_node.py` to different controllers for execution. Basic control, pan-tilt control, and visual photo taking are all completed in the same feedback loop, and there is no need to additionally start the old-version action service.

Current relevant source code paths:

```
~/microros_ws/src/multi_brains/multi_brains/llm_agent_node.py
~/microros_ws/src/multi_brains/multi_brains/controllers/motion_controller.py
~/microros_ws/src/multi_brains/multi_brains/controllers/ptz_controller.py
~/microros_ws/src/multi_brains/multi_brains/services/dify_coordinator.py
~/microros_ws/src/multi_brains/multi_brains/core/action_registry.py
```

Core actions:

```
move_forward(speed, duration)    Move forward, speed is the linear velocity,
duration is the motion time
move_backward(speed, duration)   Move backward, speed is the linear velocity,
duration is the motion time
turn_left(angle, speed)          Turn left in place, angle is the angle, speed is
the angular velocity
turn_right(angle, speed)         Turn right in place, angle is the angle, speed
is the angular velocity
servo1_move(angle)               Control servo 1 (horizontal)
servo2_move(angle)               Control servo 2 (tilt)
servo_init()                     Pan-tilt returns to the initial position
observe_surroundings()           Pan-tilt turns to multiple directions to take
photos and feeds the images back to Dify
seewhat()                        Get the current frame and feed the image back to
Dify
stop()                           Stop the chassis and the current continuous
action
```

Core flow:

```
User inputs a compound instruction
  -> Dify decision/execution layer outputs action JSON
  -> DifyCoordinator parses the response and actions
  -> llm_agent_node.py distributes actions according to action_registry
  -> MotionController publishes /cmd_vel to control the chassis
  -> PtzController publishes /cmd_ptz_angle to control the pan-tilt
  -> seewhat()/observe_surroundings() save images and upload them to Dify
  -> Dify continues to the next step or ends the task based on the action
feedback
```

`MotionController` validates parameters such as speed, duration, and turning angle to prevent abnormal values output by the large model from directly entering chassis control. `PtzController` restricts the servo angle range; by default, servo 1 ranges from -90 to 90 degrees, and servo 2 ranges from -90 to 20 degrees. When the user inputs a new instruction, `llm_agent_node.py` interrupts the current action to ensure the robot executes the latest user intent.